

表格处理

北京理工大学

表格是展示数据集的一种基本方法。本章我们介绍如何通过 Pandas 模块来处理表格。这个模块纳入了大量的库、方法和一些标准的数据类型，提供了高效地操作大型数据集所需的工具。这个模块最初是被作为金融数据分析工具而开发的，为时间序列分析提供了很好的支持。时间序列数据一般指的是对单个个体连续观测一段时间收集到的数据。但是，Pandas 这个模块的名称来自于面板数据数据分析 (Panel Data Analysis)。面板数据一般指的是我们对多个个体在一段时期内进行连续观测得到的数据。因此，在同一个时间点，我们会同时观察多个个体；对每个个体而言，我们会有多次观测。可以简单地认为，面板数据是时间序列数据的推广。

我们可以利用 Pandas 模块处理如下四种类型数据：

- ❶ 一维序列 Series：与 Numpy 模块中的 Array 非常类似。二者与 Python 基本的数据结构 List 有相似的地方，但其中一个区别是，List 中的元素可以是不同的数据类型，而 Array 和 Series 中则只允许存储相同的数据类型，这样可以更有效的使用内存，提高运算效率。
- ❷ 以时间为索引的 Time-Series：这也是一个一维序列。
- ❸ 二维表格型数据结构 DataFrame：可以将 DataFrame 理解为 Series 的容器，也就是说，多个序列 Series 放在一起，组成了 DataFrame。
- ❹ 三维面板数组结构 Panel：可以理解为 DataFrame 的容器。

我们着重介绍 Series 和 DataFrame 这两种数据结构。利用 Series 产生一维、相同类型数据；利用 DataFrame 产生二维、可以包含不同类型的数据。从这个意义上说，Series 是 DataFrame 的一种特殊情况，一个 DataFrame 中可以包含若干个 Series。为了展示方便，除 Pandas 模块以外，我们同时导入 Numpy 模块。

```
import numpy as np
import pandas as pd
```

我们导入了这些模块，并对每个模块都重新使用了更简单的名称。首先，我们调用 `Series` 函数创建一个自带索引（index）的数组。

```
s = pd.Series([1, 2, np.nan, 4])
```

我们可以通过 `print(s)` 或者直接运行 `s` 来输出结果。

```
s
```

```
0    1.0
```

```
1    2.0
```

```
2    NaN
```

```
3    4.0
```

```
dtype: float64
```

这段输出的最后一行是 Series 中数据的类型。数据在第二列输出，第一列是数据的索引 Index。我们可以分别打印出 Series 中的数值和索引：

```
s.values
```

```
array([ 1., 2., nan, 4.])
```

```
s.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

我们在介绍列表 List 的时候，List 的索引从左到右是从 0 开始的一系列整数。Series 数据在默认情况下，从上到下的索引也是如此。不过，我们在创建 Series 的时候，可以指定索引，这点和创建 Dict 是类似的。索引可以是任何类型的数据，例如：字符串。

```
s = pd.Series([1, 2, np.nan, 4],  
              index = ['A', 'B', 'C', 'D'])  
  
s
```

A 1.0
B 2.0
C NaN
D 4.0

dtype: float64

索引的目的是可以通过它来获取对应位置的数据。

```
s['A']
```

1.0

下面我们来看一下 DataFrame 的创建。DataFrame 的数据结构非常接近于电子表格或者 mysql 数据库的形式。竖列称为 columns，横行称为 index。我们可以通过 columns 和 index 来确定某个数据点的位置。我们通过一个 2×3 的矩阵来创建一个 DataFrame。

```
df = pd.DataFrame(np.arange(6).reshape(2, 3))  
df
```

	0	1	2
0	0	1	2
1	3	4	5

在创建 DataFrame 的时候可以自定义列名和索引。

```
df = pd.DataFrame(np.arange(6).reshape(2, 3),  
                  index = ['第一行', '第二行'],  
                  columns = ['A', 'B', 'C'])  
  
df
```

	A	B	C
第一行	0	1	2
第二行	3	4	5

存在多种方式创建 DataFrame。例如，在创建 DataFrame 时，可以直接指定列数据来创建 DataFrame。

```
df = pd.DataFrame({"值班": ["A", "B", "C", "D", "E"],  
                  "星期": ["一", "二", "三", "四", "五"]})  
df
```

	值班	星期
0	A	一
1	B	二
2	C	三
3	D	四
4	E	五

这是定义 DataFrame 对象的常用方法之一——使用字典。字典的“键”就是 columns 值，每个“键”对应的“值”是一个列表，是竖列中的填充数据。上面的定义中没有指定索引 (index)，所以默认的是从 0 开始的整数。

我们还可以使用“字典套字典”的方式定义 DataFrame。在字典中，可以规定列名称 (第一层键)、行索引 (第二层字典键) 以及对应的数据 (第二层字典值)，也就是在字典中规定好了每个数据格子中的数据。

```
data = {'值班': {'first': 'A', 'second': 'B'},  
        '星期': {'first': '一', 'second': '二'}}  
df = pd.DataFrame(data)  
df
```

	值班	星期
first	A	一
second	B	二

在 DataFrame 中，可以通过自定义 index 和 columns 值来修改数据顺序，这也是 columns 值和字典键明显的不同。

```
df_new = pd.DataFrame(df, index = ['second', 'first'],  
                      columns = ['星期', '值班'])  
  
df_new
```

	星期	值班
second	二	B
first	一	A

DataFrame 的不同列可以是不同的数据类型。如果以 Series 数组来创建 DataFrame，每个 Series 将成为一行，注意不是一列。

```
duty = pd.Series(['A', 'B', 'C', 'D', 'E'],  
                 index = [1, 2, 3, 4, 5])  
day = pd.Series(['一', '二', '三', '四', '五'],  
                index = [1, 2, 3, 4, 5])  
df = pd.DataFrame([duty, day])  
df
```

	1	2	3	4	5
0	A	B	C	D	E
1	一	二	三	四	五

当通过索引 (index) 或列 (columns) 来规定了相应位置上的数据时, 没有规定的位置上的值为空 (NaN)。

```
duty = pd.Series(['A', 'B', 'C', 'D', 'E'],  
                 index = [1, 2, 3, 4, 5])  
day = pd.Series(['二', '三', '四', '五'],  
                index = [2, 3, 4, 5])  
df = pd.DataFrame([duty, day])  
df
```

	1	2	3	4	5
0	A	B	C	D	E
1	NaN	二	三	四	五

接下来，我们通过几个例子来快速掌握 Pandas 模块。

首先使用字典产生数据框架。

```
NumRows = 4
df = pd.DataFrame({
    'A': pd.date_range('20191201', periods = NumRows),
    'B': pd.Series(range(NumRows), dtype = 'float'),
    'C': np.array(3, dtype = 'int32'),
    'D': pd.Categorical(['测试', '训练', '测试', '训练']),
    'E': 'foo'},
    index = list(range(NumRows)))
df
```


输出

	A	B	C	D	E
0	2019-12-01	0.0	3	测试	foo
1	2019-12-02	1.0	3	训练	foo
2	2019-12-03	2.0	3	测试	foo
3	2019-12-04	3.0	3	训练	foo

基本的描述性操作

通过 `df.dtypes()` 函数来获取数据类型。

```
df.dtypes
```

A	datetime64[ns]
B	float64
C	int32
D	category
E	object

dtype: object

通过 df.head() 函数来访问表格前几行。

```
df.head(3)
```

	A	B	C	D	E
0	2019-12-01	0.0	3	测试	foo
1	2019-12-02	1.0	3	训练	foo
2	2019-12-03	2.0	3	测试	foo

通过 df.tail() 函数来访问表格后几行。

```
df.tail(3)
```

	A	B	C	D	E
1	2019-12-02	1.0	3	训练	foo
2	2019-12-03	2.0	3	测试	foo
3	2019-12-04	3.0	3	训练	foo

输出表格的数据数目。

```
df.size
```

20

输出表格的形状。

```
df.shape
```

(4, 5)

输出表格的维度。

```
df.ndim
```

2

下面介绍输出表格中列名信息和行名信息的函数。

```
df.index
```

```
Int64Index([0, 1, 2, 3], dtype='int64')
```

```
df.columns
```

```
Index(['A', 'B', 'C', 'D', 'E'], dtype='object')
```

或者通过 `df.axes` 同时查看行名和列名信息。

```
df.axes
```

```
[Int64Index([0, 1, 2, 3], dtype='int64'), Index(['A', 'B', 'C', 'D', 'E'], dtype='object')]
```

`df.values` 用于提取表格的所有值并以数组的形式输出。

```
df.values
```

```
array([[Timestamp('2019-12-01 00:00:00'), 0.0, 3, ' 测试', 'foo'],  
       [Timestamp('2019-12-02 00:00:00'), 1.0, 3, ' 训练', 'foo'],  
       [Timestamp('2019-12-03 00:00:00'), 2.0, 3, ' 测试', 'foo'],  
       [Timestamp('2019-12-04 00:00:00'), 3.0, 3, ' 训练', 'foo']],  
      dtype=object)
```

利用 describe() 函数给出一些表格中数值的描述性统计量。

```
df.describe()
```

	B	C
count	4.000000	4.0
mean	1.500000	3.0
std	1.290994	0.0
min	0.000000	3.0
25%	0.750000	3.0
50%	1.500000	3.0
75%	2.250000	3.0
max	3.000000	3.0

当然，我们可以利用 `sum()`、`mean()`、`max()`、`min()`、`count()` 函数来单独计算某一列的统计值。

```
print(df['B'].sum())  
print(df['B'].mean())  
print(df['B'].max())  
print(df['B'].min())  
print(df['B'].count())
```

6.0

1.5

3.0

0.0

4

可以对表格进行转置操作。

df.T

	0	1	2	3
A	2019-12-01 00:00:00	2019-12-02 00:00:00	2019-12-03 00:00:00	2019-12-04 00:00:00
B	0	1	2	3
C	3	3	3	3
D	测试	训练	测试	训练
E	foo	foo	foo	foo

可以对表格按照列标签 (axis = 1) 或行标签 (axis = 0) 进行排序操作，默认是升序。

```
df.sort_index(axis = 1, ascending = False)
```

	E	D	C	B	A
0	foo	测试	3	0.0	2019-12-01
1	foo	训练	3	1.0	2019-12-02
2	foo	测试	3	2.0	2019-12-03
3	foo	训练	3	3.0	2019-12-04

也可以对表格的数值进行排序操作，通过 `by` 来指定按照某一列数值的大小顺序进行排列，默认为升序，可进行单列和多列排序。多列排序指的是当前一列相同时比较后面一列，否则优先对其前一列数值进行排序。注意，`sort_values()` 函数只能对列进行排序操作。

```
df.sort_values(by = 'B')
```

	A	B	C	D	E
0	2019-12-01	0.0	3	测试	foo
1	2019-12-02	1.0	3	训练	foo
2	2019-12-03	2.0	3	测试	foo
3	2019-12-04	3.0	3	训练	foo

排序操作

```
df.sort_values(by = ['D', 'B'], ascending = False)
```

	A	B	C	D	E
3	2019-12-04	3.0	3	训练	foo
1	2019-12-02	1.0	3	训练	foo
2	2019-12-03	2.0	3	测试	foo
0	2019-12-01	0.0	3	测试	foo

基本的选择操作

我们可以使用列标签来访问列中的数据。

```
df['A']
```

0 2019-12-01

1 2019-12-02

2 2019-12-03

3 2019-12-04

Name: A, dtype: datetime64[ns]

df.A 等价于 df['A']

df.A

0 2019-12-01

1 2019-12-02

2 2019-12-03

3 2019-12-04

Name: A, dtype: datetime64[ns]

我们可以通过下标来访问指定的行。

```
df[0:3]
```

	A	B	C	D	E
0	2019-12-01	0.0	3	测试	foo
1	2019-12-02	1.0	3	训练	foo
2	2019-12-03	2.0	3	测试	foo

按照标签进行混合选择操作

DataFrame 提供了两个操作符来访问其中的数据。

- ❶ loc: 通过行和列的索引来访问数据;
- ❷ iloc: 通过行和列的下标来访问数据。

下面来看几个例子。

按照标签选择指定行。

```
df.loc[0, :]
```

A	2019-12-01 00:00:00
B	0.0
C	3
D	测试
E	foo

Name: 0, dtype: object

按照标签选择指定列。

```
df.loc[:, ['A', 'B']]
```

	A	B
0	2019-12-01	0.0
1	2019-12-02	1.0
2	2019-12-03	2.0
3	2019-12-04	3.0

按照标签选择指定的行和列。首先输出表格的前三行以及 A、B 列。

```
df.loc[0:3, ['A', 'B']]
```

	A	B
0	2019-12-01	0.0
1	2019-12-02	1.0
2	2019-12-03	2.0
3	2019-12-04	3.0

下面来看几个关于 `iloc` 的例子。

按照行下标选择.

```
df.iloc[3, :]
```

A 2019-12-04 00:00:00

B 3

C 3

D 训练

E foo

Name: 3, dtype: object

按照行以及列的下标选择

```
df.iloc [1:3, 0:2]
```

	A	B
1	2019-12-02	1.0
2	2019-12-03	2.0

按照行以及列的下标选择

```
df.iloc [[1, 2], [0, 2]]
```

	A	C
1	2019-12-02	3
2	2019-12-03	3

按照行以及列的下标选择

```
df.iloc [1:2, :]
```

	A	B	C	D	E
1	2019-12-02	1.0	3	训练	foo

按照行以及列的下标选择

```
df.iloc[:, 1:5]
```

	B	C	D	E
0	0.0	3	测试	foo
1	1.0	3	训练	foo
2	2.0	3	测试	foo
3	3.0	3	训练	foo

按照行以及列的下标选择

```
df.iloc[1, 1]
```

1.0

我们顺便介绍一个函数：iat。df.iat[1, 1] 这个命令尽管与 df.iloc[1, 1] 等价，但是运行速度比 df.iloc[1, 1] 快。

```
df.iat[1,1]
```

1.0

根据布尔索引选取

我们打算访问具有指定特征的行。下面看几个例子：

```
df[df.B > 1]
```

	A	B	C	D	E
2	2019-12-03	2.0	3	测试	foo
3	2019-12-04	3.0	3	训练	foo

根据布尔条件对整个表格进行筛选，返回与表格形状相同的 dataframe，符合条件的位置返回原始数值，否则为空 (NaN)。

```
df_sub = df.iloc[0 : 2, 1 : 3]  
df_sub
```

	B	C
0	0.0	3
1	1.0	3

接上页

```
df_sub[df_sub > 0]
```

	B	C
0	NaN	3
1	1.0	3

利用 isin() 函数进行过滤。

```
df[df['D'].isin(['测试'])]
```

	A	B	C	D
0	2019-12-01	0.0	3	测试
2	2019-12-03	2.0	3	测试

我们可以对 dataframe 进行增加一列或者修改指定位置的值的操作。来看下面的例子。

```
s1 = pd.Series([1, 2, 3, 4], index = [0, 1, 2, 3])
```

```
s1
```

```
0    1
```

```
1    2
```

```
2    3
```

```
3    4
```

```
dtype: int64
```

例子

```
df['F'] = s1  
df
```

	A	B	C	D	E	F
0	2019-12-01	0.0	3	测试	foo	1
1	2019-12-02	1.0	3	训练	foo	2
2	2019-12-03	2.0	3	测试	foo	3
3	2019-12-04	3.0	3	训练	foo	4

下面是修改表格中指定位置的值。

```
df.loc[0, 'C'] = 0  
df
```

	A	B	C	D	E	F
0	2019-12-01	0.0	0	测试	foo	1
1	2019-12-02	1.0	3	训练	foo	2
2	2019-12-03	2.0	3	测试	foo	3
3	2019-12-04	3.0	3	训练	foo	4

修改表格中指定位置的值

```
df.loc[1, "C"]= 0  
df
```

	A	B	C	D	E	F
0	2019-12-01	0.0	0	测试	foo	1
1	2019-12-02	1.0	0	训练	foo	2
2	2019-12-03	2.0	3	测试	foo	3
3	2019-12-04	3.0	3	训练	foo	4

修改表格中指定位置的值

```
df.loc[:, 'G'] = np.array([5] * len(df))
df
```

	A	B	C	D	E	F	G
0	2019-12-01	0.0	0	测试	foo	1	5
1	2019-12-02	1.0	0	训练	foo	2	5
2	2019-12-03	2.0	3	测试	foo	3	5
3	2019-12-04	3.0	3	训练	foo	4	5

修改表格中指定位置的值

```
dfcopy = df.copy()
dfcopy[dfcopy['C'] > 0]
```

		A	B	C	D	E	F	G
2	2019-12-03	2.0	3	测试	foo	3	5	
3	2019-12-04	3.0	3	训练	foo	4	5	

索引可以自定义，自定义的索引会自动寻找原来的索引，如果是一样的，就取原来索引对应的值，这个可以简称为“自动对齐”。reindex() 可以改变、增加或者删减一些索引值，并返回一个副本。

```
df_rd1 = df.reindex(columns = ['A', 'D', 'H', 'F'])  
df_rd1
```

	A	D	H	F
0	2019-12-01	测试	NaN	1
1	2019-12-02	训练	NaN	2
2	2019-12-03	测试	NaN	3
3	2019-12-04	训练	NaN	4

当重新设置的索引与被操作的 dataframe 的索引不一致时，其值被 NaN 填充。如
果不想以 NaN 填充，则可以利用 fill_value 来设置填充方式。

```
df_rd2 = df.reindex(columns = ['A', 'D', 'H', 'F'],  
                    fill_value = 14)  
  
df_rd2
```

	A	D	H	F
0	2019-12-01	测试	14	1
1	2019-12-02	训练	14	2
2	2019-12-03	测试	14	3
3	2019-12-04	训练	14	4

通过 method 的方法来设置向前填充 (method = 'ffill' 或 'pad') 以及向后填充 (method = 'bfill' 或 'backfill')。向前填充, 指在重新设置的索引没有相对应的数值时取前一个索引的值作为填充。向后填充, 指在重新设置的索引没有相对应的数值时取后一个索引的值作为填充。

```
df_rd3 = df.reindex(columns = ['A', 'D', 'H', 'F'],  
                    method = 'ffill')  
df_rd3
```

	A	D	H	F
0	2019-12-01	测试	5	1
1	2019-12-02	训练	5	2
2	2019-12-03	测试	5	3
3	2019-12-04	训练	5	4

我们可以通过 `drop()` 函数实现删除某些特定是行 (`axis = 0`) 和列 (`axis = 1`)。默认值 `axis=0`，也就是按照行删除。

```
df.drop([0, 1], inplace = True)
df
```

	A	B	C	D	E	F	G
2	2019-12-03	2.0	3	测试	foo	3	5
3	2019-12-04	3.0	3	训练	foo	4	5

按照列标签来删除表格中多余的列

```
df.drop('G', axis = 1, inplace = True)  
df
```

	A	B	C	D	E	F
2	2019-12-03	2.0	3	测试	foo	3
3	2019-12-04	3.0	3	训练	foo	4

前面是按照列标签来删除表格中多余的列。如果要删除多列的话，建议按序号来删除列。

```
delCol = [4, 5]  
df.drop(df.columns[delCol], axis = 1, inplace = True)  
df
```

	A	B	C	D
2	2019-12-03	2.0	3	测试
3	2019-12-04	3.0	3	训练

当删除多余的行或者列时，会导致相应的索引变得不连续。此时，可以指定时间序列为索引列，或者可以指定某一特定的列为索引列。看下面这个例子。

```
df.set_index('A', inplace = True)  
df
```

	B	C	D
A			
2019-12-03	2.0	3	测试
2019-12-04	3.0	3	训练

例子

```
df.reset_index(level = [0], inplace = True)
```

```
df
```

	A	B	C	D
0	2019-12-03	2.0	3	测试
1	2019-12-04	3.0	3	训练

为了争夺欧洲霸权，1812 年夏，拿破仑集结军队，渡过涅曼河，向俄国不宣而战。关于这场战争，可以参考百度百科网页：[https://baike.baidu.com/item/俄法 1812 年战争](https://baike.baidu.com/item/俄法%201812%20年战争)。这里，我们主要学习利用 Python 进行表格处理。首先，读入数据。在这个例子中，我们把数据保存在一个叫做 Minard.csv 的文件中。大家可以把这个数据记录下来，然后保存为 csv 格式，更改一下所在文件的目录即可。

```
minard = pd.read_csv('~ / Minard.csv') # ~ 请指定文件所在的路径
minard
```

	longitude	latitude	city	direction	survivors
0	32.0	54.8	Smolensk	Advance	145000
1	33.2	54.9	Dorogobouge	Advance	140000
2	34.4	55.5	Chjat	Advance	127100
3	37.6	55.8	Moscou	Advance	100000
4	34.3	55.2	Wixma	Retreat	55000
5	32.0	54.6	Smolensk	Retreat	24000
6	30.4	54.4	Orscha	Retreat	20000
7	26.8	54.3	Moiodexno	Retreat	12000

检查表格大小。

```
[len(minard.index), len(minard.columns)]
```

[8, 5]

注：可以比较一下 `minard.shape`。

表格列标签。

```
minard.columns
```

```
Index(['longitude', 'latitude', 'city', 'direction', 'survivors'], dtype='object')
```

改列名称。

```
minard.rename(columns={'city': 'city name'}, inplace = True)
```

```
minard
```

	longitude	latitude	city name	direction	survivors
0	32	54.8	Smolensk	Advance	145000
1	33.2	54.9	Dorogobouge	Advance	140000
2	34.4	55.5	Chjat	Advance	127100
3	37.6	55.8	Moscou	Advance	100000
4	34.3	55.2	Wixma	Retreat	55000
5	32	54.6	Smolensk	Retreat	24000
6	30.4	54.4	Orscha	Retreat	20000
7	26.8	54.3	Moiodexno	Retreat	12000

提取单列。

```
minard['survivors']
```

0	145000
1	140000
2	127100
3	100000
4	55000
5	24000
6	20000
7	12000

Name: survivors, dtype: int64

提取单列

```
minard.loc[:, 'survivors']
```

0	145000
1	140000
2	127100
3	100000
4	55000
5	24000
6	20000
7	12000

Name: survivors, dtype: int64

提取单列

```
minard.iloc[:, 4]
```

0	145000
1	140000
2	127100
3	100000
4	55000
5	24000
6	20000
7	12000

Name: survivors, dtype: int64

提取单列

```
minard.iloc[0, 4]
```

145000

```
minard.iloc[5, 4]
```

24000

```
initial = minard.iloc[0, 4]
```

```
initial
```

145000

列操作。

```
percent_surviving = minard.iloc[:, 4]/ initial  
percent_surviving
```

```
0    1.000000  
1    0.965517  
2    0.876552  
3    0.689655  
4    0.379310  
5    0.165517  
6    0.137931  
7    0.082759
```

Name: survivors, dtype: float64

列操作

```
minard['percent surviving'] = percent_surviving
```

```
minard
```

	longitude	latitude	city	direction	survivors	percent surviving
0	32	54.8	Smolensk	Advance	145000	1.000000
1	33.2	54.9	Dorogobouge	Advance	140000	0.965517
2	34.4	55.5	Chjat	Advance	127100	0.876552
3	37.6	55.8	Moscou	Advance	100000	0.689655
4	34.3	55.2	Wixma	Retreat	55000	0.37931
5	32	54.6	Smolensk	Retreat	24000	0.165517
6	30.4	54.4	Orscha	Retreat	20000	0.137931
7	26.8	54.3	Moiodexno	Retreat	12000	0.082759

接下来，我们一起学习如何添加%。

```
minard['percent surviving'] = pd.Series(["{0:.2f}%".format(val * 100) for val in minard['percent surviving']])
minard
```

	longitude	latitude	city	direction	survivors	percent surviving
0	32	54.8	Smolensk	Advance	145000	100.00%
1	33.2	54.9	Dorogobouge	Advance	140000	96.55%
2	34.4	55.5	Chjat	Advance	127100	87.66%
3	37.6	55.8	Moscou	Advance	100000	68.97%
4	34.3	55.2	Wixma	Retreat	55000	37.93%
5	32	54.6	Smolensk	Retreat	24000	16.55%
6	30.4	54.4	Orscha	Retreat	20000	13.79%
7	26.8	54.3	Moiodexno	Retreat	12000	8.28%

接上页

```
minard[['longitude', 'latitude']]
```

	longitude	latitude
0	32	54.8
1	33.2	54.9
2	34.4	55.5
3	37.6	55.8
4	34.3	55.2
5	32	54.6
6	30.4	54.4
7	26.8	54.3

下面我们介绍如何删除表格中的数据。drop 方法并不实际改变数据框，而是提取被删除数据后剩下的部分。可以通过 `help(minard.drop)` 查询 drop 方法中各参数的具体含义。与 drop 方法不同的是，del 方法会直接删除数据框中指定的数据。

```
minard1=minard.drop(minard[['longitude', 'latitude', 'direction']],  
                    axis = 1,  
                    inplace = False)  
minard1
```

	city	survivors	percent surviving
0	Smolensk	145000	100.00%
1	Dorogobouge	140000	96.55%
2	Chjat	127100	87.66%
3	Moscou	100000	68.97%
4	Wixma	55000	37.93%
5	Smolensk	24000	16.55%
6	Orscha	20000	13.79%
7	Moiodexno	12000	8.28%

对比 drop() 函数，del 函数功能如下：

```
del minard['longitude']
```

```
minard
```

	latitude	city	direction	survivors	percent surviving
0	54.8	Smolensk	Advance	145000	100.00%
1	54.9	Dorogobouge	Advance	140000	96.55%
2	55.5	Chjat	Advance	127100	87.66%
3	55.8	Moscou	Advance	100000	68.97%
4	55.2	Wixma	Retreat	55000	37.93%
5	54.6	Smolensk	Retreat	24000	16.55%
6	54.4	Orscha	Retreat	20000	13.79%
7	54.3	Moiiodexno	Retreat	12000	8.28%

国外某地区人口资料

读入数据,

	年龄	性别	A	B	C
0	合计	小计	423198	1568985	37228
1	合计	男	212946	808256	18751
2	合计	女	210252	760729	18477
3	1-30 岁	小计	90193	435430	20102
4	1-30 岁	男	47583	240425	10288
5	1-30 岁	女	42610	195005	9814
6	31-60 岁	小计	200535	663479	9381
7	31-60 岁	男	100045	333349	4719
8	31-60 岁	女	100490	330130	4662
9	60 岁以上	小计	132470	470076	7745
10	60 岁以上	男	65318	234482	3744
11	60 岁以上	女	67152	235594	4001

计算此表格的行数与列数

```
print(pop.shape[0], pop.shape[1], len(pop))
```

12 5 12

因此，此表格有 12 行，5 列。下面列出此表格的 column 名称、性别列中的所有值和年龄列中的所有值。

```
pop.columns
```

```
Index(['年龄', '性别', 'A', 'B', 'C'], dtype='object')
```

```
pop['性别'].unique()
```

```
array(['小计', '男', '女'], dtype=object)
```

接上页

```
pop['年龄'].unique()
```

```
array(['合计', '1-30 岁', '31-60 岁', '60 岁以上'], dtype=object)
```

大的表格包含了许多信息，我们只提取其中一部分数据来分析，比如，我们挑选 A 和 B 地区的人口数据来分析。

```
partial_pop = pop[['年龄', '性别', 'A', 'B']]
```

我们来看看 A 和 B 的人口差值。

```
change = partial_pop['A'] - partial_pop['B']
```

在表格中添加一列。

```
partial_pop['Change'] = change  
partial_pop.head(3)
```

	年龄	性别	A	B	Change
0	合计	小计	423198	1568985	-1145787
1	合计	男	212946	808256	-595310
2	合计	女	210252	760729	-550477

计算一下两地人口差别的百分比，并按照指定格式输出。

```
partial_pop['Percent Change'] = change/partial_pop['A']  
partial_pop['Percent Change'] = pd.Series(pd.Series(["{0:.2f}%".format(val * 100)  
for val in partial_pop['Percent Change']]))  
partial_pop.head(3)
```

	年龄	性别	A	B	Change	Percent Change
0	合计	小计	423198	1568985	-1145787	-270.74%
1	合计	男	212946	808256	-595310	-279.56%
2	合计	女	210252	760729	-550477	-261.82%

复制生成一个新的表格。

```
diff_pop= partial_pop.copy()
```

按照人口变化数量的降序排序。

```
diff_pop1 = diff_pop.sort_values(by = 'Change', ascending = False)  
diff_pop1.head(5)
```

	年龄	性别	A	B	Change	Percent Change
5	1-30 岁	女	42610	195005	-152395	-357.65%
11	60 岁以上	女	67152	235594	-168442	-250.84%
10	60 岁以上	男	65318	234482	-169164	-258.99%
4	1-30 岁	男	47583	240425	-192842	-405.27%
8	31-60 岁	女	100490	330130	-229640	-228.52%

从表中数据可以看出，所有年龄段的人口 A 地区都低于 B 地区。

为了研究 B 地区的男女人口比例，我们需要进行一系列操作。

```
diff_pop1_B = diff_pop1.drop(['A', 'Change', 'Percent Change'],  
                             axis = 1)  
  
diff_pop1_B.tail(3)
```

	年龄	性别	B
2	合计	女	760729
1	合计	男	808256
0	合计	小计	1568985

下面我们来计算男女性别比例。通过对 'B' 列进行排序，从而新产生一个数据框架。

```
all_ages = diff_pop1_B[diff_pop1_B['年龄'] == '合计']
all_ages = all_ages.sort_values(by = 'B', ascending = False)
all_ages = pd.DataFrame(all_ages.values,
                        columns = all_ages.columns)
all_ages.head()
```

	年龄	性别	B
0	合计	小计	1568985
1	合计	男	808256
2	合计	女	760729

性别比例按照指定格式输入。

```
all_ages['Proportion'] = all_ages['B'] / all_ages.loc[0, 'B']  
all_ages['Proportion'] = pd.Series(["{0: .2f}%".format(val * 100)  
for val in all_ages['Proportion']])  
all_ages
```

	年龄	性别	B	Proportion
0	合计	小计	1568985	100.00%
1	合计	男	808256	51.51%
2	合计	女	760729	48.49%

下面我们计算 B 地区 1-30 岁的男女比例。首先提取 B 地区年龄为 1-30 岁的人口数。

```
young = diff_pop1_B[diff_pop1_B['年龄']=='1-30岁']
```

新产生一个数据框架。

```
young = young.sort_values(by = 'B', ascending = False)
young = pd.DataFrame(young.values, columns = young.columns)
young.head()
```

	年龄	性别	B
0	1-30 岁	小计	435430
1	1-30 岁	男	240425
2	1-30 岁	女	195005

计算性别比例并按照指定格式输出。

```
young['Proportion'] = young['B']/young.loc[0, 'B']
young['Proportion'] = pd.Series(["{0:.2f}%".format(val * 100)
for val in young['Proportion']])
young
```

	年龄	性别	B	Proportion
0	0 岁	小计	435430	100.00%
1	0 岁	男	240425	55.22%
2	0 岁	女	195005	44.78%

下面我们来关注 B 地区各年龄段男女人口比例。首先提取女性人口数据。

```
females_all_rows = diff_pop1_B[diff_pop1_B['性别'] == '女']
```

把各个年龄段的女性总人口剔除。

```
females = females_all_rows[females_all_rows['年龄'] != '合计']  
females = females.sort_index(ascending = True,  
                             axis = 0, inplace = False)
```

同样的可以得到男性人口数据。

```
males_all_rows = diff_pop1_B[diff_pop1_B['性别'] == '男']  
males = males_all_rows[males_all_rows['年龄'] != '合计']  
males = males.sort_index(ascending = True,  
                         axis = 0, inplace = False)
```

下面计算男女性别比例。

```
ratios = np.array(females['B']) / np.array(males['B'])  
ratios = pd.DataFrame({'年龄': np.array(males['年龄']),  
                        'B:F:M RATIO': ratios})
```

```
ratios.sort_index(ascending = False, axis = 1,  
                  inplace = False).head(3)
```

	年龄	B:F:M RATIO
0	0.811085	1-30 岁
1	0.990343	31-60 岁
2	1.004742	60 岁以上